

LAB: TensorFlow Tutorial

TensorFlow Graph

The entire purpose of TensorFlow is to have a so-called computational graph that can be executed much more efficiently than if the same calculations were to be performed directly in Python. TensorFlow can be more efficient than NumPy because TensorFlow knows the entire computation graph that must be executed, while NumPy only knows the computation of a single mathematical operation at a time. TensorFlow can also automatically calculate the gradients that are needed to optimize the variables of the graph so as to make the model perform better. This is because the graph is a combination of simple mathematical expressions so the gradient of the entire graph can be calculated using the chain-rule for derivatives.

TensorFlow can also take advantage of multi-core CPUs as well as GPUs - and Google has even built special chips just for TensorFlow which are called TPUs (Tensor Processing Units) and are even faster than GPUs.

A TensorFlow graph consists of the following parts which will be detailed below:

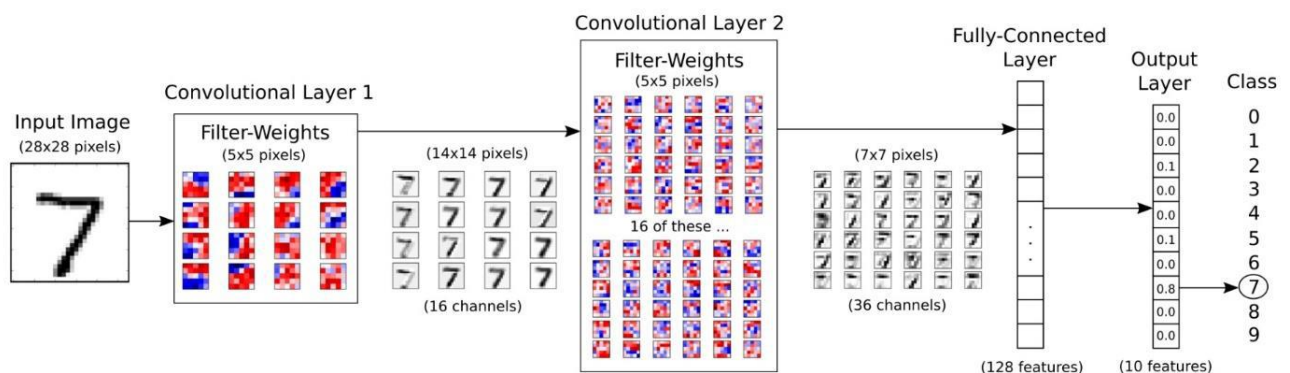
- **Placeholder variables** used for inputting data to the graph.
- **Variables that are going to be optimized** so as to make the convolutional network perform better.
- **The mathematical formulas for the convolutional network** (Building the model).
- **A cost measure** that can be used to guide the optimization of the variables.
- **An optimization method** which updates the variables.

The database and problem formulation

The famous MNIST data-set contains images of hand-written digits from 0 to 9. In this tutorial, we will implement a simple Convolutional Neural Network in TensorFlow which has a classification accuracy of about 99%.

Understanding the CNN architecture

The following chart shows roughly how the data flows in the Convolutional Neural Network that is implemented below.



A convolution layer comprises convolution step, max pooling step and applying the rectifier function step.

1. The image input size is 28x28 pixels. We are going to use 16 filters such as the size of each filter is 5x5 pixels. How many feature maps (channels) do we obtain from a single image?
2. We use a pooling matrix of size 2x2. What is the resulting size of the images?
Note that, in the convolution step, the stride is set to 1 (x-axis and y-axis). The size of the output after the convolution step is kept the same by padding the image with zeroes.
3. The output of the first convolutional layer is then passed to a second convolutional layer. For each new image (feature map) we want to apply a filter and we want to have 36 output channels. How many filters do we need at the second convolution layer?
4. We use a pooling matrix of size 2x2 also at the second convolution layer. What is the resulting size of the images?
5. How many nodes does the flatten layer contain?
6. We use only one hidden layer in the ANN composed of 128 neurons. How many nodes should we have at the output layer?

Imports

Before you start, upload in colab all the .py files (cahe, dataset, download and mnist)

```
import matplotlib.pyplot as plt
import tensorflow.compat as tf
import numpy as np
from sklearn.metrics import confusion_matrix
import time
from datetime import timedelta
import math
```

Configuration of Neural Network

```
# Convolutional Layer 1.
filter_size1 = 5 # Convolution filters are 5 x 5 pixels.
num_filters1 = 16 # There are 16 of these filters.
# Convolutional Layer 2.
filter_size2 = 5 # Convolution filters are 5 x 5 pixels.
num_filters2 = 36 # There are 36 of these filters.
# Fully-connected layer.
fc_size = 128 # Number of neurons in fully-connected layer.
```

Load the data

```
from keras import layers, models

# Load MNIST dataset
data = tf.keras.datasets.mnist
(x_train, y_train), (x_test, y_test) = data.load_data()
# Convert the input data to numpy arrays
x_train = np.array(x_train)
y_train = np.array(y_train)
x_test = np.array(x_test)
y_test = np.array(y_test)
```

```
# Normalize the input data by dividing by 255
x_train = x_train / 255.0
x_test = x_test / 255.0
```

- 1- What is the size of the training and test sets?
- 2- Why is the value of the `img_size_flat` equal to 784? The `num_channels` is equal to 1 what does that mean?

```
# Get the number of training and testing samples
num_train = x_train.shape[0]
num_test = x_test.shape[0]
print("Number of training samples:", num_train)
print("Number of testing samples:", num_test)
```

```
image = x_train[0] # Example: Using the first image from the MNIST training set
```

```
# Get the size of the image
image_height = image.shape[0]
image_width = image.shape[1]
```

```
print("Image dimensions (height x width):", image_height, "x", image_width)
```

```
# Assuming image_shape represents the shape of each image in the dataset
img_shape = (image_height, image_width)
num_channels = 1 # Grayscale images have one channel
img_size_flat = image_height * image_width * num_channels
print("Size of flattened image:", img_size_flat)
```

Helper-function for plotting images

Function used to plot 9 images in a 3x3 grid, and writing the true and predicted classes below each image.

```
def plot_images(images, cls_true, cls_pred=None):
    assert len(images) == len(cls_true) == 9

    # Create figure with 3x3 sub-plots.
    fig, axes = plt.subplots(3, 3)
    fig.subplots_adjust(hspace=0.3, wspace=0.3)

    for i, ax in enumerate(axes.flat):
        # Plot image.
        ax.imshow(images[i].reshape(img_shape), cmap='binary')

        # Show true and predicted classes.
        if cls_pred is None:
            xlabel = "True: {0}".format(cls_true[i])
        else:
            xlabel = "True: {0}, Pred: {1}".format(cls_true[i], cls_pred[i])

        # Show the classes as the label on the x-axis.
        ax.set_xlabel(xlabel)

        # Remove ticks from the plot.
        ax.set_xticks([])
        ax.set_yticks([])

    # Ensure the plot is shown correctly with multiple plots
    # in a single Notebook cell.
    plt.show()
```

Then Plot a few images to see if data is correct by executing the following:

```
# Get the first images from the test-set.
images = data.x_test[0:9]
# Get the true classes for those images.
cls_true = data.y_test_cls[0:9]

# Plot the images and labels using our helper-function above.
plot_images(images=images, cls_true=cls_true)
```

- 1- Show the class names and number of classes
- 2- Build your CNN using the parameters from the start of the lab